Welcome

Digital and Computer Vision

Lesson 3: Session 3

Object Detection with Haar Cascades

Summary of the **previous** session: Feature Detection with OpenCV

In the previous session we have covered :

What are features on images?
Detecting key points using Harris Corner Detection.
Feature matching (ORB, SIFT, and SURF).

Summary of the current session: Object Detection with Haar Cascades

In this session we will covered the following:

Understanding Haar Cascades for face detection.
Object detection with HOG-SVM algorithm
Digits classification with HOG-SVM algorithm

Exercise from previous session 1/4

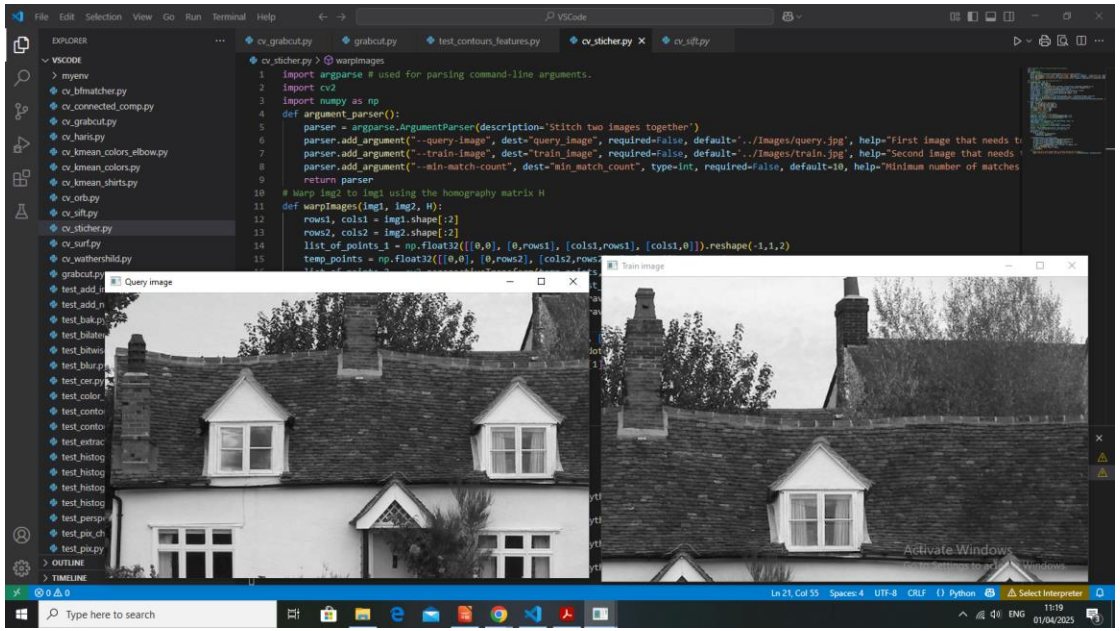
In the book of Jodseph Howse, Prateek Joshi, Michael Beyeler, “Open CV Computer Vision Project with Python”, read the Chapter 5: Creating a Panoramic Image. Explain and run the code for creating panoramic images from this chapter

```
import argparse # used for parsing command-line arguments.
import cv2
import numpy as np
def argument_parser():
    parser = argparse.ArgumentParser(description='Stitch two images together')
    parser.add_argument("--query-image", dest="query_image", required=False,
default='../Images/query.jpg', help="First image that needs to be stitched")
    parser.add_argument("--train-image", dest="train_image", required=False,
default='../Images/train.jpg', help="Second image that needs to be stitched")
    parser.add_argument("--min-match-count", dest="min_match_count", type=int,
required=False, default=10, help="Minimum number of matches required")
    return parser
```

Exercise from previous session 2/4

```
# Warp img2 to img1 using the homography matrix H
def warpImages(img1, img2, H):
    rows1, cols1 = img1.shape[:2]
    rows2, cols2 = img2.shape[:2]
    list_of_points_1 = np.float32([[0,0], [0,rows1], [cols1,rows1], [cols1,0]]).reshape(-1,1,2)
    temp_points = np.float32([[0,0], [0,rows2], [cols2,rows2], [cols2,0]]).reshape(-1,1,2)
    list_of_points_2 = cv2.perspectiveTransform(temp_points, H)
    list_of_points = np.concatenate((list_of_points_1, list_of_points_2), axis=0)
    [x_min, y_min] = np.int32(list_of_points.min(axis=0).ravel() - 0.5)
    [x_max, y_max] = np.int32(list_of_points.max(axis=0).ravel() + 0.5)
    translation_dist = [-x_min,-y_min]
    H_translation = np.array([[1, 0, translation_dist[0]], [0, 1, translation_dist[1]], [0,0,1]])
    output_img = cv2.warpPerspective(img2, H_translation.dot(H), (x_max-x_min, y_max-y_min))
    output_img[translation_dist[1]:rows1+translation_dist[1], translation_dist[0]:cols1+translation_dist[0]] = img1
    return output_img
```

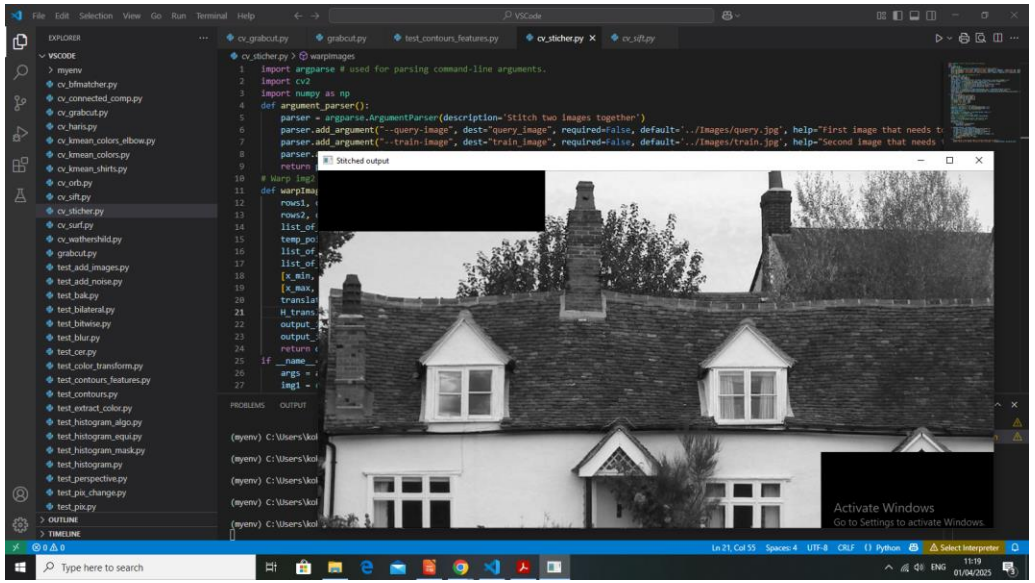
Exercise from previous session 3/4




```
if __name__=='__main__':
    args = argument_parser().parse_args()
    img1 = cv2.imread(args.query_image, 0)
    img2 = cv2.imread(args.train_image, 0)
    min_match_count = args.min_match_count
    cv2.imshow('Query image', img1)
    cv2.imshow('Train image', img2)
    # Initialize the SIFT detector
    sift = cv2.SIFT_create()
    # Extract the keypoints and descriptors
    keypoints1, descriptors1 = sift.detectAndCompute(img1, None)
    keypoints2, descriptors2 = sift.detectAndCompute(img2, None)
    # Initialize parameters for Flann based matcher
    FLANN_INDEX_KDTREE = 0
    index_params = dict(algorithm = FLANN_INDEX_KDTREE, trees = 5)
    search_params = dict(checks = 50)
# Initialize the Flann based matcher object
    flann = cv2.FlannBasedMatcher(index_params, search_params)
```

Exercise from previous session 4/4

```
    # Compute the matches
matches = flann.knnMatch(descriptors1, descriptors2, k=2)
# Store all the good matches as per Lowe's ratio test
good_matches = []
for m1,m2 in matches:
    if m1.distance < 0.7*m2.distance:
        good_matches.append(m1)
if len(good_matches) > min_match_count:
    src_pts = np.float32([ keypoints1[good_match.queryIdx].pt
    for good_match in good_matches ]).reshape(-1,1,2)
    dst_pts = np.float32([ keypoints2[good_match.trainIdx].pt
    for good_match in good_matches ]).reshape(-1,1,2)
    M, mask = cv2.findHomography(src_pts, dst_pts, cv2.RANSAC, 5.0)
    result = warpImages(img2, img1, M)
    cv2.imshow('Stitched output', result)
    cv2.waitKey()
else:
    print ("We don't have enough number of matches between the two images.")
    print ("Found only %d matches. We need at least %d matches." % len(good_matches), min_match_count)
```



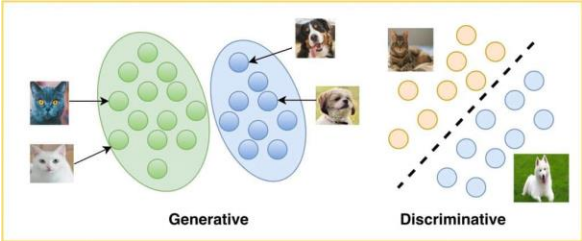
Classification and Generation Algorithms

Computer vision involve two major types of ML models:

1. **Classification Algorithms**: Identify and categorize an image into a predefined class.

How it Works:

A classifier is trained on a dataset of labeled images (e.g., cats vs. dogs).
The model extracts features (edges, textures, shapes) and learns patterns.
When given a new image, the classifier predicts the most likely category.



https://learnopencv.com/generative-and-discriminative-models/

2. **Generation Algorithms** : generate entirely new images from training data.

How it Works:

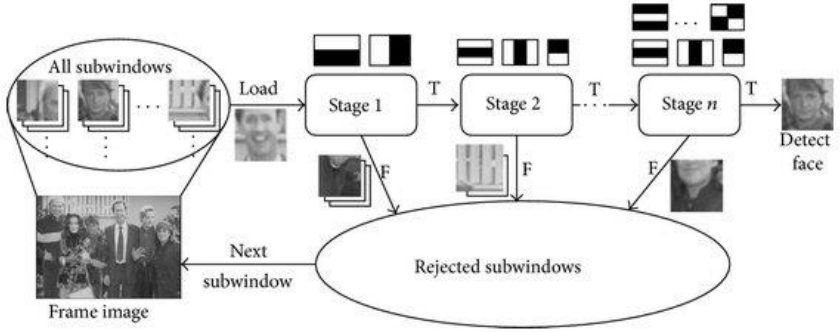
Given a dataset of images, the model is trained to learns patterns such as shapes, colors, and textures.

Produce new samples based on these models and additional constrained information (prompt)

Haar Cascades Classifier

Haar cascades are a machine learning-based approach for object detection, commonly used for face, eye, and body detection in images and videos. This algorithm were proposed by Paul Viola and Michael Jones in their paper, "Rapid Object Detection using a Boosted Cascade of Simple Features" in 2001.

Why cascades ?
Multi-Stage Filtering (Cascade Architecture) : Instead of analyzing every region of an image with a complex classifier, Haar Cascade uses a series of simple classifiers arranged in stages. If a region fails at an early stage, it is immediately discarded. This saves computational time and makes the algorithm fast and efficient.

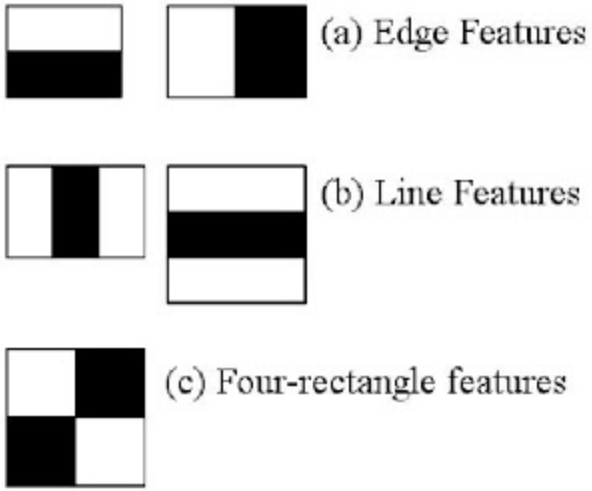


https://machinelearningknowledge.ai/face-detection-with-haar-cascade-in-opencv-python/

Haar-like features and Haar cascades

We need **features**, producing stable classification and tracking from the image data. Each Haar-like feature describes the pattern of contrast among adjacent image regions. They are just like our convolutional kernel. Each feature is a single value obtained by subtracting sum of pixels under the white rectangle from sum of pixels under the black rectangle.

Two images that differ only in scale should be capable of yielding similar features, albeit for different window sizes. Thus, it is useful to detect features for multiple window sizes. Such a collection of features is detected in Haar **cascade**. We may say a Haar cascade is scale-invariant or, in other words, robust to changes in scale.



Fast computation of Haar features using integral images

The problem is that we have many various kernel sizes and locations to extract features. For each feature calculation, we must compute the sum of pixels under the white and black rectangles. To reduce these computations, the Haar Cascade algorithm uses Integral Images, as described in the previous section.

```
[[1, 0, 0, 1, 0],
 [0, 0, 0, 1, 0],
 [0, 1, 0, 1, 0],
 [1, 1, 0, 1, 0],
 [1, 0, 0, 0, 1]]
```

Original image

```
[[ 1,  1,  1,  2,  2],
 [ 1,  1,  1,  3,  3],
 [ 1,  2,  2,  5,  5],
 [ 2,  4,  4,  8,  8],
 [ 3,  5,  5,  9, 10]]
```

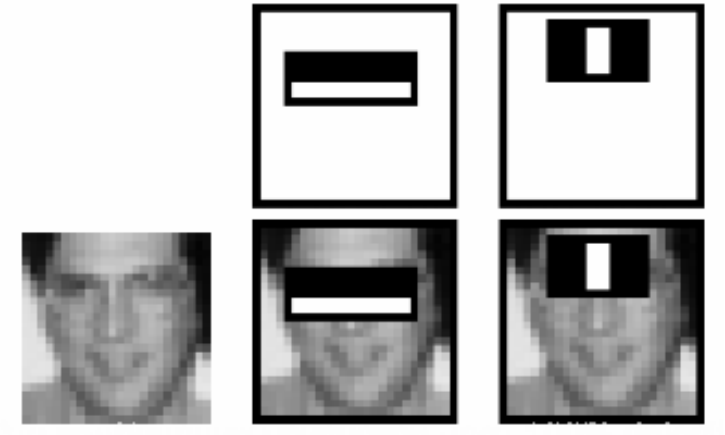
Integral image

https://theailearner.com/2019/08/24/integral-images/

Sum = 8 + 1 - 2 - 2 = 5

Sum = Bottom right + top left – top right – bottom left

Selecting good features using AdaBoost algorithm



https://docs.opencv.org/3.4/db/d28/tutorial_cascade_classifier.html

AdaBoost (Adaptive Boosting) is a machine learning algorithm that combines multiple weak classifiers into a strong classifier.

Haar cascades use thousands of Haar-like features to detect objects (e.g., faces). However, most of these features are not useful, and using all of them would be computationally expensive.

AdaBoost helps by:

Selecting only the most important features.
Combining weak classifiers into a strong classifier.

How Does AdaBoost Work in Haar Cascades?

1. **Initialize Weights**: Each training image is given an equal weight.

2. **Train Weak Classifiers**: Each Haar feature is tested to see how well it classifies an object (e.g., face vs. non-face).

3. **Evaluate Errors**: The classifier with the lowest error gets a higher weight. Misclassified sample images get higher weights, so they are given more attention in the next round. The process is continued until the required accuracy or error rate is achieved or the required number of features are found).

5. **Combine Weak Classifiers**: A final strong classifier is built using a weighted sum of all weak classifiers.

Summarising Haar Cascades Algorithm

1. **Train on Positive and Negative Images** : Positive images contain the object (e.g., faces). Negative images contain anything except the object.

2. **Haar Features** : Uses Haar-like features, which are patterns of dark and light regions.

3. **Integral Images** : Speeds up feature computation by using a summed-area table to quickly calculate pixel sums.

4. **Adaboost Algorithm** : Selects the most important features while eliminating redundant ones, improving efficiency.

5. **Cascade of Classifiers** : Multiple classifiers are arranged in stages. If an object is detected in all stages, it is classified as a face (or the intended object).

Haar Cascades in OpenCV

OpenCV provides a training method or pretrained models, that can be read using the cv2.CascadeClassifier() method. Detection is done using the detectMultiScale() method with parameters :

scaleFactor→ How much the image size is reduced at each scale (1.1 means a 10% reduction).

minNeighbors → Minimum neighboring rectangles needed to confirm a detection.

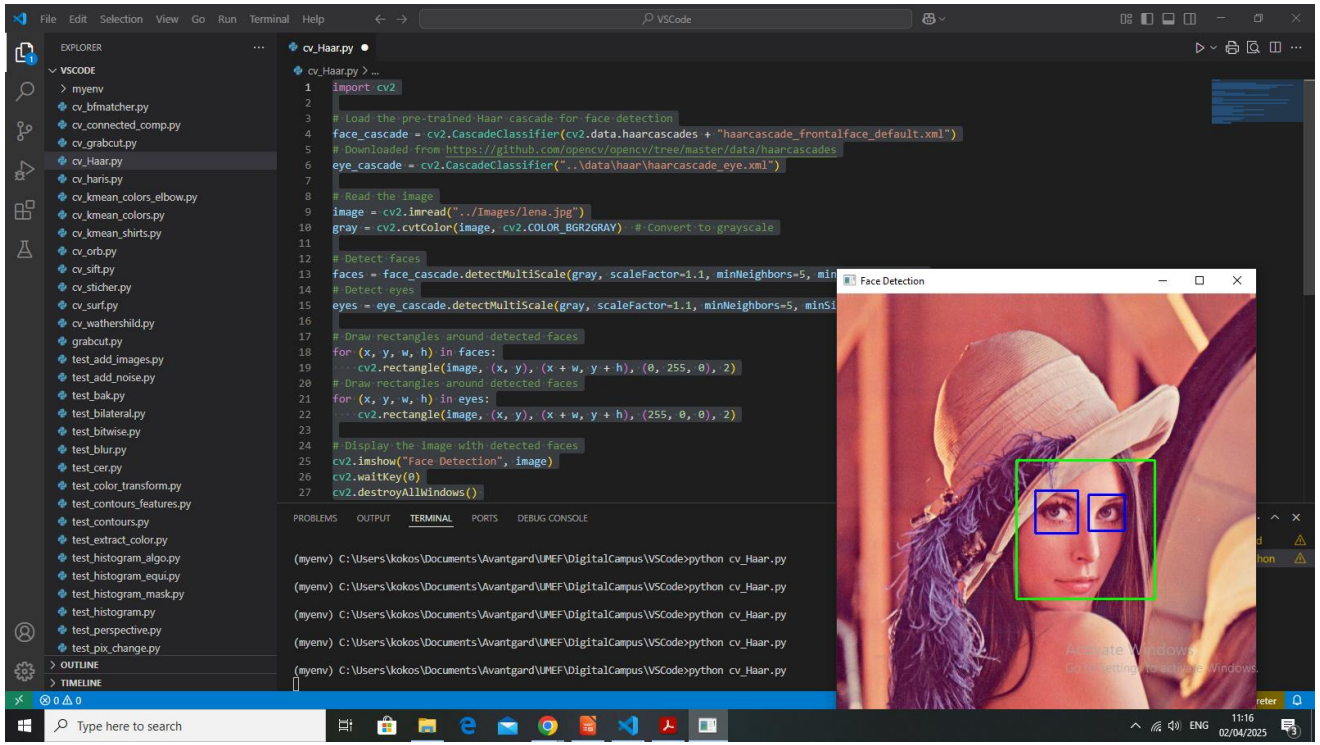
MinSize → Minimum size of objects to detect.

Haar Cascades code example 1/2

```
import cv2

# Load the pre-trained Haar cascade for face detection
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
# Downloaded from https://github.com/opencv/opencv/tree/master/data/haarcascades
eye_cascade = cv2.CascadeClassifier("..\Models\Haarcascade\haarcascade_eye.xml")

# Read the image
image = cv2.imread("../Images/lena.jpg")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)  # Convert to grayscale

# Detect faces
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
# Detect eyes
eyes = eye_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
```

Haar Cascades code example 2/2

```
# Draw rectangles around detected faces
for (x, y, w, h) in faces:
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
# Draw rectangles around detected faces
for (x, y, w, h) in eyes:
    cv2.rectangle(image, (x, y), (x + w, y + h), (255, 0, 0), 2)

# Display the image with detected faces
cv2.imshow("Face Detection", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Haar Cascades example with life video stream

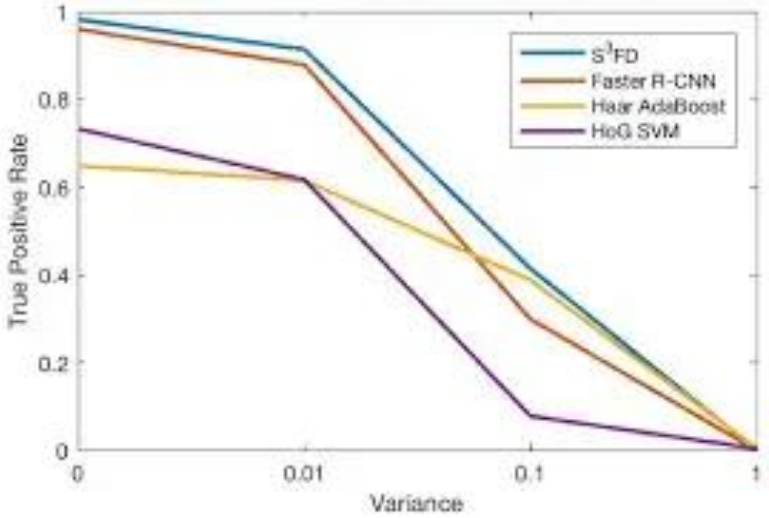
Let's now change the code to capture video stream and detect faces in real time. We can also make some tests of the robustness of the algorithm. We will do this directly in VSCode editor :

Haar Cascades vs CNN

1. **Training Data** – HAAR Cascade requires comparatively fewer data in order to train whereas CNN requires thousands of images per class to achieve respectable accuracy.

2. **Training and Inference (detection) times** : Since training is based on scanning multiple feature windows across different scales, it takes a long time (often hours or days). Once trained, Haar cascades run relatively fast in real-time. CNNs require training on a large labeled dataset using backpropagation and gradient descent. This requires powerful GPUs and takes time (hours to days) but results in a highly optimized and generalized face detector. Once trained, CNNs can process an image much faster than Haar cascades.

3. **Accuracy** – Both CNN and HAAR Cascade classifiers show reasonably good accuracies. But due to revolutionary ongoing research in deep learning, the accuracy of CNN models is almost tending towards 100% now.



https://machinelearningknowledge.ai/face-detection-with-haar-cascade-in-opencv-python/

Training Haar Cascades in OpenCV

OpenCV provides tools to train a Haar cascade classifier, but the process is computationally expensive and requires a well-prepared dataset.

1. Prepare the Dataset
You need two sets of images:

Positive images: Contain the object you want to detect (e.g., faces).
Negative images: Do not contain the object (random backgrounds).

Dataset Guidelines: Positive images should have the object well-aligned and clearly visible. Negative images should be diverse and should not contain the object. The more images, the better the classifier.

2. Create a Positive Samples File
To generate samples, use OpenCV's opencv_createsamples tool: opencv_createsamples
This generates synthetic positive samples from one object image.

3. Train the Classifier
Use opencv_traincascade to train the Haar cascade classifier. Training takes time! It can take hours or even days depending on dataset size.

Problems training Haar cascades in OpenCV

Apart the very longer training times Haar cascade training is not available in the last version of OpenCV (the one we are using). The reason is that it was considered that Haar cascades have to be replaced with more efficient Deep Learning methods.

To train Haar cascades we have to download the source code of version 3.4... from the Github and recompile it for python. Then we have to instal this version in a new virtual environment, which clearly go out of the subject of this course

HOG + SVM for Object Detection in OpenCV

HOG (Histogram of Oriented Gradients) with SVM (Support Vector Machine) is a classical yet effective method for object detection, particularly in detecting pedestrians and other rigid objects. It is widely used due to its simplicity and decent accuracy without requiring deep learning.

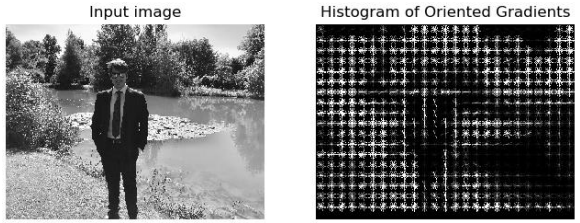
As usual in CV tasks we need features and that the part of HOG. Then we needs a clasificator this is SVM ML algorithm.

Understanding HOG (Histogram of Oriented Gradients)

HOG is a feature descriptor used in computer vision and image processing for object detection. It works by:

1. Dividing the image into small connected regions (cells).
2. Computing gradient orientation histograms in each cell.
3. Grouping cells into larger spatial blocks and normalizing the histograms.
4. Concatenating all the histograms to form the final HOG descriptor.

Why use HOG?
HOG captures edge orientations and gradient information, which helps in detecting objects irrespective of illumination and contrast changes. Works well for detecting rigid objects like pedestrians and vehicles.



https://github.com/SamPlvs/Object-detection-via-HOG-SVM

Understanding SVM (Support Vector Machine)

SVMs are binary classifiers (can be used to learn two categories) but can be extended to be used in **multiclass** classification.

SVM (Support Vector Machine) is a supervised learning algorithm used for classification.

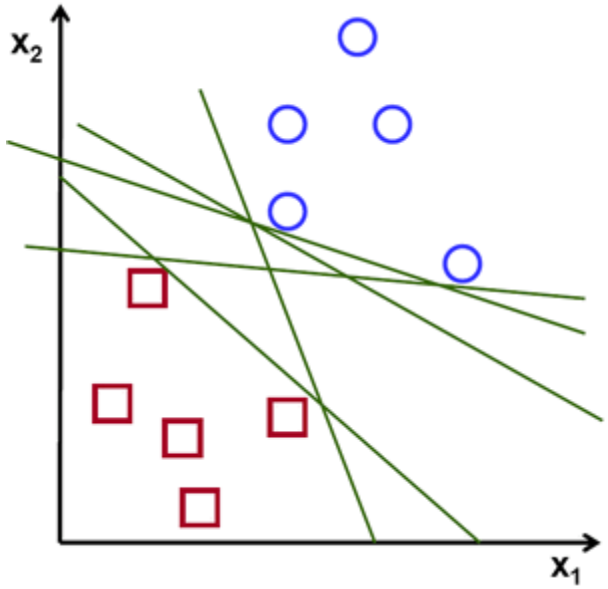
It tries to find the optimal hyperplane that separates object and non-object samples.

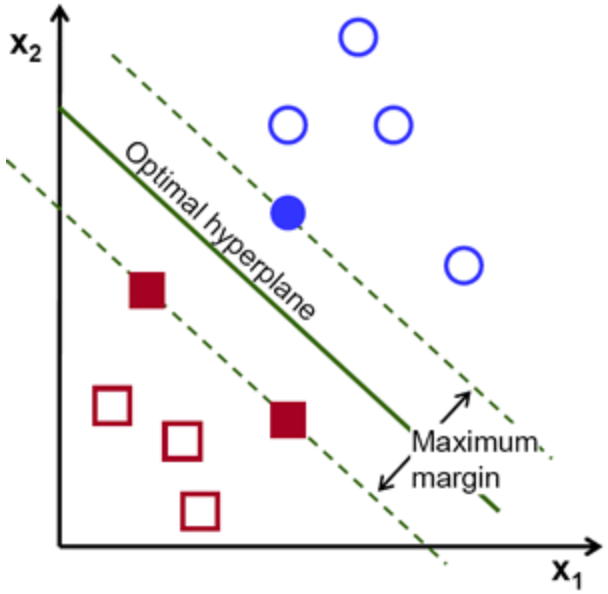
It performs well for binary classification tasks like "pedestrian vs. non-pedestrian."

SVM in its simplest form, is a linear discriminator which optimal hyperplane that best separates data points of different classes in a feature space. It maximizes the margin between the closest points (support vectors) of each class. However, SVM can also be non-linear when using kernel tricks to project data into a higher-dimensional space where it becomes linearly separable.

Linearly Separable Data

Consider the image below which has two types of data, red and blue. Such data which can be divided into two with a straight line (or hyperplanes in higher dimensions) is called Linear Separable.You can see plenty of such lines are possible. Which one we will take? Very intuitively we can say that the line should be passing as far as possible from all the points. Why? Because there can be noise in the incoming data. This data should not affect the classification accuracy. So taking a farthest line will provide more immunity against noise. So what SVM does is to find a straight line with largest minimum distance to the training samples.

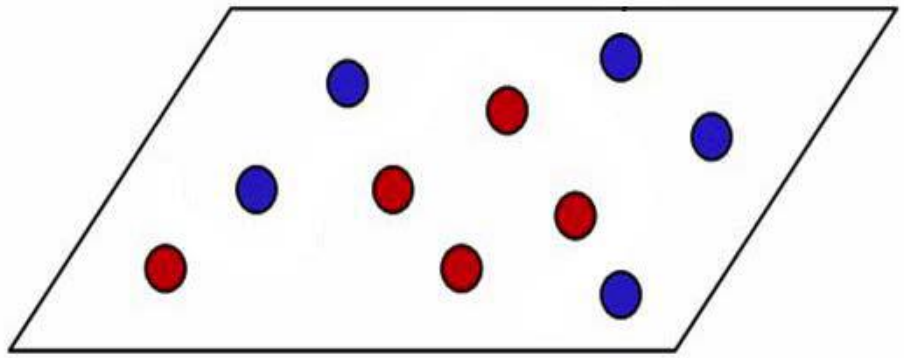


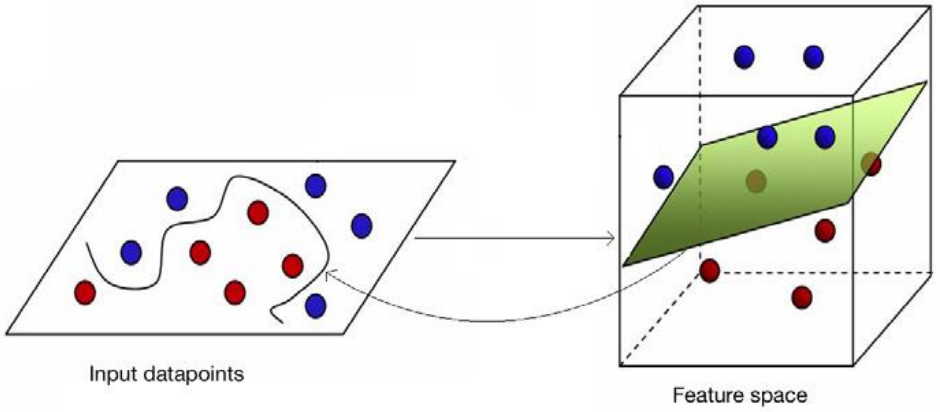


https://docs.opencv.org/4.x/d4/db1/tutorial_py_svm_basics.html

Nonlinearly Separable Data

As we can see, we cannot draw a simple straight line to separate the red points from the blue points. Coming up with a nice curvy boundary that will satisfy all the points is prohibitively expensive. SVMs are really good at drawing "straight lines". So, what's our answer here? The good thing about SVMs is that they can draw these "straight lines" in any number of dimensions. So technically, if you project these points into a high dimensional space, where they can separated by a simple hyperplane, SVMs will come up with an exact boundary. Once we have that boundary, we can project it back to the original space. The projection of this hyperplane on our original lower dimensional space looks curvy, as we can see in the figure right.

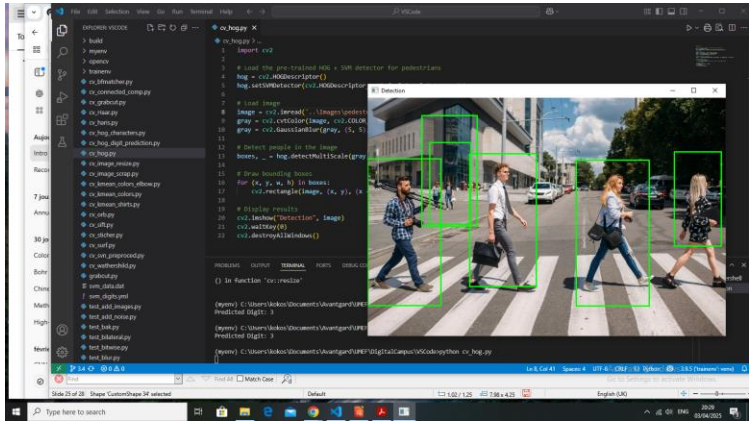




Joseph Howse, Prateek Joshi, Michael Beyeler
OpenCV: Computer Vision Projects with Python

HOG-SVM example code

```
import cv2

# Load the pre-trained HOG + SVM detector for pedestrians
hog = cv2.HOGDescriptor()
hog.setSVMDetector(cv2.HOGDescriptor_getDefaultPeopleDetector())
# Load image
image = cv2.imread('..\Images\pedestrian.jpg')
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
gray = cv2.GaussianBlur(gray, (5, 5), 0)
# Detect people in the image
boxes, _ = hog.detectMultiScale(gray, winStride=(2, 2), padding=(8, 8), scale=1.1)
# Draw bounding boxes
for (x, y, w, h) in boxes:
    cv2.rectangle(image, (x, y), (x + w, y + h), (0, 255, 0), 2)
# Display results
cv2.imshow("Detection", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



HOG-SVM pros and cons

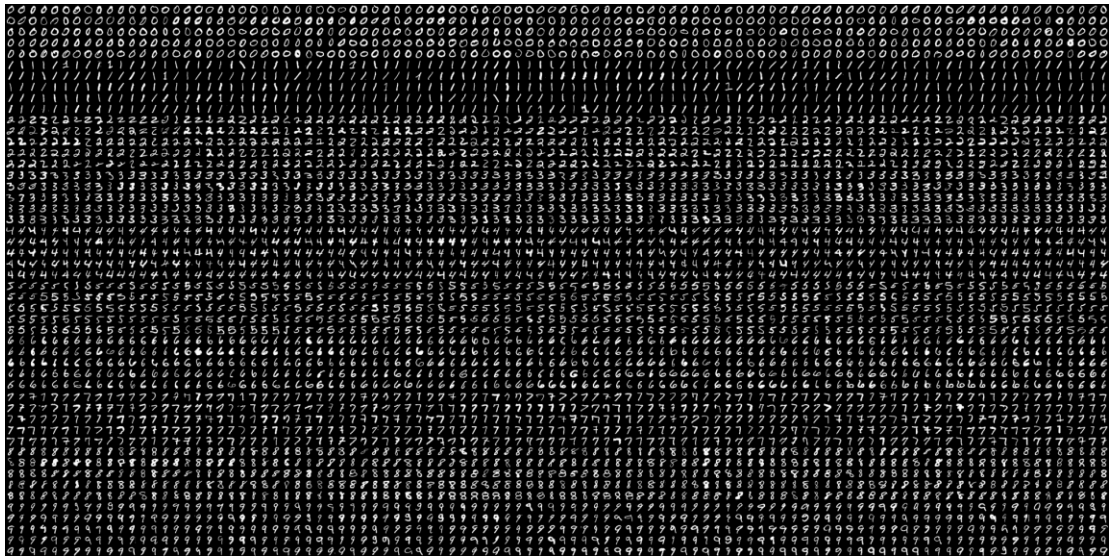
Pros	Cons
Works well for detecting rigid objects	Struggles with highly deformable objects
Does not require deep learning	Not as accurate as modern CNN-based models
Faster than deep learning methods	Requires manual feature extraction
Works well on CPUs	More false positives compared to YOLO/SSD

Training HOG-SVM algorithm for classification of handwritten digits

Our goal is to build an application which can read handwritten digits. For this we need some training data and some test data. OpenCV comes with an image digits.png (in the folder opencv/samples/data/) which has 5000 handwritten digits (500 for each digit). Each digit is a 20x20 image.

Training steps are :

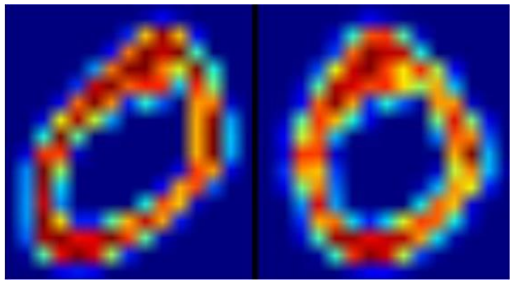
1. Split this image into 5000 different digit images.
2. For each digit (20x20 image) we will use Histogram of Oriented Gradients (HOG) as feature vectors.
3. We train the SVM algorithm with 250 of images for each digit and use the other 250 for testing.



https://docs.opencv.org/4.x/dd/d3b/tutorial_py_svm_opencv.html

Training HOG-SVM algorithm for classification of handwritten digits – code example 1/4

```
import cv2 as cv
import numpy as np

SZ=20
bin_n = 16 # Number of bins
 affine_flags = cv.WARP_INVERSE_MAP|cv.INTER_LINEAR
# Takes a digit image and deskew it.
def deskew(img):
    gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
    _, binary = cv.threshold(gray, 127, 255, cv.THRESH_BINARY)
    m = cv.moments(binary)
    if abs(m['mu02']) < 1e-2:
        return img.copy()
    skew = m['mu11']/m['mu02']
    M = np.float32([[1, skew, -0.5*SZ*skew], [0, 1, 0]])
    img = cv.warpAffine(img,M,(SZ, SZ),flags=affine_flags)
    return img
```



https://docs.opencv.org/4.x/dd/d3b/tutorial_py_svm_opencv.html

Training HOG-SVM algorithm for classification of handwritten digits – code example 2/4

```
# Find the HOG Descriptor of each cell. For that, we find Sobel derivatives of each cell in X and Y direction.
# Then find their magnitude and direction of gradient at each pixel. This gradient is quantized to 16 integer values.
# Divide this image to four sub-squares. For each sub-square, calculate the histogram of direction (16 bins) weighted
with their magnitude.
# So each sub-square gives you a vector containing 16 values. Four such vectors (of four sub-squares) together gives
us a feature vector containing 64 values.

def hog(img):
    gx = cv.Sobel(img, cv.CV_32F, 1, 0)
    gy = cv.Sobel(img, cv.CV_32F, 0, 1)
    mag, ang = cv.cartToPolar(gx, gy)
    bins = np.int32(bin_n*ang/(2*np.pi))    # quantizing binvalues in (0...16)
    bin_cells = bins[:10,:10], bins[10:,:10], bins[:10,10:], bins[10:,10:]
    mag_cells = mag[:10,:10], mag[10:,:10], mag[:10,10:], mag[10:,10:]
    hists = [np.bincount(b.ravel(), m.ravel(), bin_n) for b, m in zip(bin_cells, mag_cells)]
    hist = np.hstack(hists)     # hist is a 64 bit vector
    return hist
```

Training HOG-SVM algorithm for classification of handwritten digits – code example 3/4

```
# read the image and divide to 5000 subimages 20x20 pixels each
img = cv.imread('..\Images\digits.png')
if img is None:
    raise Exception("we need the digits.png image from samples/data here !")
cells = [np.hsplit(row,100) for row in np.vsplit(img,50)]

# First half is trainData, remaining is testData
train_cells = [ i[:50] for i in cells ]
test_cells = [ i[50:] for i in cells]

deskewed = [list(map(deskew,row)) for row in train_cells]
hogdata = [list(map(hog,row)) for row in deskewed]
# Reshape data to have feature vector containing 64 float values
trainData = np.float32(hogdata).reshape(-1,64)
responses = np.repeat(np.arange(10),250)[:,np.newaxis]
```

Training HOG-SVM algorithm for classification of handwritten digits – code example 4/4

```
# define and train the SVM model
svm = cv.ml.SVM_create()
svm.setKernel(cv.ml.SVM_LINEAR)   # use linear model
svm.setType(cv.ml.SVM_C_SVC)        #  type of SVM algorithm used : Multi-class classification
svm.setC(2.67)  #The C parameter controls regularization how much the model should avoid misclassifications.
svm.setGamma(5.383) #his parameter is used in non-linear kernels (not used in our case
 svm.train(trainData, cv.ml.ROW_SAMPLE, responses)
# save the obtained model for next use
svm.save('svm_data.dat')

# we do the same for the test data and compute the correct %
deskewed = [list(map(deskew,row)) for row in test_cells]
hogdata = [list(map(hog,row)) for row in deskewed]
testData = np.float32(hogdata).reshape(-1,bin_n*4)
result = svm.predict(testData)[1]
 mask = result==responses
correct = np.count_nonzero(mask)
print(correct*100.0/result.size)
```

Detecting handwritten digits with HOG-SVM model saved previously – code example 1/4

Once we have trained a model we can use it to recognise digits in text. Below is a code example :

```
import cv2 as cv
import numpy as np
SZ = 20  # Size to which images will be resized
bin_n = 16  # Number of bins
affine_flags = cv.WARP_INVERSE_MAP | cv.INTER_LINEAR
# Function to deskew the images
def deskew(img):
    if len(img.shape) == 3:  # If the image has more than one channel
        gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
    else:
        gray = img  # If it's already grayscale, use as-is
    _, binary = cv.threshold(gray, 127, 255, cv.THRESH_BINARY)
    m = cv.moments(binary)
    if abs(m['mu02']) < 1e-2:
        return img.copy()
    skew = m['mu11'] / m['mu02']
    M = np.float32([[1, skew, -0.5 * SZ * skew], [0, 1, 0]])
    img = cv.warpAffine(img, M, (SZ, SZ), flags=affine_flags)
    return img
```

Detecting handwritten digits with HOG-SVM model saved previously – code example 2/4

```
# Function to extract HOG features
def hog(img):
    gx = cv.Sobel(img, cv.CV_32F, 1, 0)
    gy = cv.Sobel(img, cv.CV_32F, 0, 1)
    mag, ang = cv.cartToPolar(gx, gy)
    bins = np.int32(bin_n * ang / (2 * np.pi))  # quantizing binvalues in (0...16)
    bin_cells = bins[:10, :10], bins[10:, :10], bins[:10, 10:], bins[10:, 10:]
    mag_cells = mag[:10, :10], mag[10:, :10], mag[:10, 10:], mag[10:, 10:]
    hists = [np.bincount(b.ravel(), m.ravel(), bin_n) for b, m in zip(bin_cells, mag_cells)]
    hist = np.hstack(hists)  # hist is a 64 bit vector
    return hist
```

Detecting handwritten digits with HOG-SVM model saved previously – code example 3/4

```
# Function to preprocess a single digit image (resize and deskew)
def preprocess_digit(image_path):
    img = cv.imread(image_path, cv.IMREAD_GRAYSCALE)
    if img is None:
        print("Error loading image!")
        return None
    img = cv.resize(img, (SZ, SZ))  # Resize to 20x20
    img = deskew(img)  # Deskew the image if necessary
    img = img.astype(np.uint8)  # Convert to uint8 type for HOGDescriptor

    # Extract HOG features
    hog_features = hog(img)

    return np.float32(hog_features).reshape(1, -1)  # Reshape for SVM input
```

Detecting handwritten digits with HOG-SVM model saved previously – code example 4/4

```
# Function to recognize a single digit
def recognize_digit(image_path):
    # Preprocess the input image and extract HOG features
    hog_features = preprocess_digit(image_path)
    if hog_features is None:
        print("Failed to process the input image.")
        return None
    # Load the pre-trained SVM model
    svm = cv.ml.SVM_load('svm_data.dat')
    # Use the SVM to predict the digit
    result = svm.predict(hog_features)[1]

    return int(result.item()) # Use item() to extract the scalar value

# the program starts
image_path = '..\Images\dig_7.png'  # Replace with your image file path
predicted_digit = recognize_digit(image_path)
print(f"Predicted Digit: {predicted_digit}")
```

Comparison between Haar cascades and HOG-SVM

Feature	Haar Cascades	HOG + SVM
Feature Extraction	Haar-like features (edges, lines, rectangles)	Histogram of Oriented Gradients (HOG)
Classifier	AdaBoost cascade	Support Vector Machine (SVM)
Training Complexity	Requires large datasets & long training time	Faster training with fewer samples
Detection Speed	Very fast (real-time capable)	Slower than Haar but still efficient
Accuracy & Robustness	High false positives, sensitive to lighting/scale	More robust, fewer false positives
Best for	Face, eye, hand detection	Pedestrians, digits, general object recognition
Use Case Preference	Works well for structured objects with clear edges	Better for objects with complex variations

Which One to Use?

Use Haar Cascades when:

Speed is a priority (e.g., real-time face detection).
You have a well-structured object with clear edges.
You have a large dataset for training.

Use HOG + SVM when:

You need higher accuracy and fewer false positives.
You are detecting objects with more complex shapes (e.g., pedestrians, digits).
You want a model that generalizes better across different conditions.

Exercise

Study and explain the k-Nearest Neighbors (KNN) algorithm. Then, implement a Python program to train a classifier for handwritten digits using KNN.

Additionally, compare the performance and characteristics of KNN with the HOG-SVM approach. Discuss their strengths, weaknesses, and suitability for different applications.

Use the web resources provided below for reference.

https://docs.opencv.org/4.x/d5/d26/tutorial_py_knn_understanding.html

In this session we have covered:

Object Detection with Haar Cascades
Understanding Haar Cascades for face detection.
Object detection with HOG-SVM algorithm
Digits classification with HOG-SVM algorithm

In the next session we will covered:

Morphological Operations
Dilation, Erosion, Opening, and Closing.
Removing noise using morphological transformations.

THANKS !!!